



Laboratory Activity No. 2

From Syntax to Semantics: Building a Grammar Checker and Evaluator

Name(s):	Section:
Liezel Anne R. Guim Rechelle Casinto Von Eiron Abion Lorenz Felonia	BSCS 4-1
Group No. / Team:	Date:
Group 1	9/14/2026

Reflections:

1. In your own words, what is the difference between what your Task A code checks and what your Task B code checks?

Task A doesn't care about the answer; it only checks whether the input string is valid according to the grammar. It checks the structure of the expression by making sure it follows the rules of `<expr>`, `<term>`, and `<factor>`. If something doesn't fit the grammar, such as a stray `+` or a missing `)`, the parser reports an error and stops.

Task B focuses on the meaning of the expression. Once the syntax is confirmed to be valid, it calculates the actual numerical value. The code ensures that an answer is only computed after the syntax check succeeds, so an invalid expression is never evaluated. This reflects the difference between syntax (whether it is correctly written) and semantics (what it means or evaluates to).

2. Point to the exact place in your code where recursion happens, and explain which grammar rule it corresponds to.

The recursion happens in the `parse_factor()` function at line 116, where `parse_factor()` calls `parse_expr()` when it encounters an opening parenthesis `(`. This allows the parser to process the entire expression inside the parentheses before continuing with the rest of the expression. After parsing the inner expression, the program checks for the required closing parenthesis `)`. This corresponds to the grammar rule `<factor> -> ( <expr> ) | <digit>`. For example, when processing `(3+4)*2`, `parse_factor()` calls `parse_expr()` to evaluate `3+4`, returns the value `7`, and then `parse_term()` continues by multiplying it by `2`.

3. Explain, using your `2+3*4` example, why ambiguous grammar is a real problem and not just a theoretical one.

The ambiguous grammar is a real problem because it can cause the same input to produce different results. In the program, the `naive_left_to_right_evaluation()` function deliberately ignores operator precedence. For `2+3*4`, it first calculates `2+3`,





giving 5, and then multiplies by 4, producing 20.

However, the corrected parser produces 14 because `parse_expr()` handles `+` and `-`, while `parse_term()` handles `*` and `/`. Therefore, $3*4$ is calculated before $2+...$. The program demonstrates both results in `demonstrate_ambiguity()`.

This shows that ambiguity is not merely theoretical because different interpretations of the same string can produce different numerical meanings. The grammar needs to establish precedence so the parser has one intended interpretation.

4. If you were to add a new operator (for example, exponentiation $^$) to this grammar, where in the `expr/term/factor` hierarchy would it need to go, and why?

If we were to add a new operator, such as exponentiation $^$, to the grammar, we would place it in a new level between `term` and `factor`, creating a `power` level. This is because exponentiation normally has a higher precedence than multiplication and division. The hierarchy would become `expr()` for addition and subtraction, `term()` for multiplication and division, `power()` for exponentiation, and `factor()` for digits and parentheses. This structure would ensure that an expression such as $2*3^2$ is interpreted as $2*(3^2) = 18$ rather than $(2*3)^2 = 36$. In the current code, `parse_term()` calls `parse_factor()` to establish the precedence of multiplication and division over addition and subtraction, so adding a `power()` level would similarly establish the higher precedence of exponentiation.

```
File Edit Selection View Go Run Terminal Help python
Problems Output Debug Console Terminal Ports
PS C:\Users\LiezGo\Documents\Programming Languages\python> python syntax_semantics.py
=== Ambiguity Demonstration ===
Expression: 2+3*4
Left-to-right evaluation: 20
Precedence-based evaluation: 14
The unfixed grammar (<expr> -> <expr> + <expr> | <expr> * <expr> | <digits>) gives '+' and '*' equal status, so '2+3*4' has two legal parse trees. The corrected grammar removes the ambiguity by placing
<term> below <expr> in the hierarchy, forcing '*'/'/' to bind to their neighboring <factor>s before any '+'/'-' at the <expr> level applies, and making the left-to-right repetition at each level give l
eft-associativity, so there's exactly one parse tree per string.

=== Required Test Cases ===
3+4*2      -> Valid -> 11
(3+4)*2    -> Valid -> 14
8/2-1      -> Valid -> 3
3+4         -> Invalid
(3+4        -> Invalid
2+3*4       -> Valid -> 14

Enter expression: 3+4*2
Valid syntax
Result: 11
Enter expression: (3+4)*2
Valid syntax
Result: 14
Enter expression: 8/2-1
Valid syntax
Result: 3
Enter expression: 3+4
Invalid syntax
Error at position 2: unexpected token '+'
Enter expression: (3+4
Invalid syntax
Error at position 5: missing closing parenthesis ')'
Enter expression: 2+3*4
Valid syntax
Result: 14
Enter expression: quit
Program finished.
PS C:\Users\LiezGo\Documents\Programming Languages\python>
```

